

SWE 4101

Object Oriented Concepts I

Objectives

01

Encapsulation



✓ Encapsulation (Recap)

- ✓ • Encapsulation is one of the four pillars of OOP
- ✓ • A means of protecting the class, the integrity of its data and behavior
- ✓ • Bundles data (variables) and methods (functions) into a single unit, such as a class, while **restricting direct access to some of an object's components**

A Banking Scenario

- Consider a small retail bank that needs a software model for a customer's account
- The bank's first business rules are:
 1. Every account has a unique account number assigned by the bank, never by the customer
 2. Every account must have an owner and a currency at the moment it is opened
 3. The balance of any account must never be negative

A Banking Scenario

```
public class BankAccount {  
    public String accountNumber;  
    public String ownerName;  
    public double balance;  
    public String currency;  
}
```

A Banking Scenario

4. Can deposit or withdraw money but the amount should be positive

```
public class BankAccount {  
    public String accountNumber;  
    public String ownerName;  
    public double balance;  
    public String currency;  
    public void deposit(double amount) {  
        balance = balance + amount;  
    }  
  
    public void withdraw(double amount)  
    {  
        balance = balance - amount;  
    }  
}
```

A Banking Scenario

Before making EVERYTHING private, ask these questions:

- Which fields, if any, should the outside world be able to modify directly?
- Which should it be able to read but not modify?
- Which should it not be able to see at all?

✓ A Banking Scenario

- balance → must never be negative
- accountNumber → must always be valid and unchangeable
- currency → must always exist and stay fixed
- ownerName → must always exist and be validated before change

Making fields 'private'

```
public class BankAccount {  
    private String accountNumber;  
    private String ownerName;  
    private double balance;  
    private String currency;  
    public void deposit(double amount) {  
        balance = balance + amount;  
    }  
  
    public void withdraw(double amount)  
    {  
        balance = balance - amount;  
    }  
}
```

✓ The Role of Constructors

- The default no-argument constructor instantiates an object with default values
- It will produce an account with a null accountNumber, a null owner, a null currency, and a zero balance
- Is this object valid according to the rules?
- That is a broken object: there cannot be a half-built Bank Account!

A Parameterized Constructor

Hence, we need to write a parameterized constructor to create a valid object

```
public BankAccount(String accountNumber, String ownerName, String
currency, double openingDeposit) {
    this.accountNumber = accountNumber;
    this.ownerName = ownerName;
    this.currency = currency;
    this.balance = openingDeposit;
}
```

```
BankAccount a = new BankAccount(null, "", null, -3.5);
```

← Can create invalid object

Adding Validation

```
public BankAccount(String accountNumber, String ownerName, String currency, double
openingDeposit) {
    These if statements check whether everything is valid

    if (accountNumber == null || accountNumber.isBlank()) {
        throw new IllegalArgumentException("Account number is required.");
    }
    if (ownerName == null || ownerName.isBlank()) {
        throw new IllegalArgumentException("Owner name is required.");
    }
    if (currency == null || currency.isBlank()) {
        throw new IllegalArgumentException("Currency is required.");
    }
    if (openingDeposit < 0.0) {
        throw new IllegalArgumentException("Opening deposit cannot be negative.");
    }

    this.accountNumber = accountNumber;
    this.ownerName = ownerName;
    this.currency = currency;
    this.balance = openingDeposit;
}
```

Question

Real-world banks open accounts in several distinct ways:

-  ■ An account with no opening deposit
-  ■ An account opened with a cash deposit
-  ■ An account migrated from a partner bank with an existing balance

How do we incorporate this?

✓ Constructor Overloading

```
public BankAccount(String accountNumber, String ownerName, String currency) {  
  
    if (accountNumber == null || accountNumber.isBlank()) {  
        throw new IllegalArgumentException("Account number is required.");  
    }  
    if (ownerName == null || ownerName.isBlank()) {  
        throw new IllegalArgumentException("Owner name is required.");  
    }  
    if (currency == null || currency.isBlank()) {  
        throw new IllegalArgumentException("Currency is required.");  
    }  
  
    this.accountNumber = accountNumber;  
    this.ownerName = ownerName;  
    this.currency = currency;  
    this.balance = 0.0;  
}
```

How do we get rid of so much duplication?

✓ Constructor Chaining

- ✓ The process of calling one constructor from another within the same class
- ✓ Helps in reducing code duplication and ensuring that all object initialization logic is centralized and consistent
- ✓ Done using **this()**

✓ Constructor Chaining

```
public class Student {  
  
    private String name;  
    private int age;  
    private double cgpa;  
  
    // Main constructor  
    public Student(String name, int age, double cgpa) {  
        this.name = name;  
        this.age = age;  
        this.cgpa = cgpa;  
    }  
  
    public Student(String name) {  
        this(name, 18, 0.0);  
    }  
  
    public Student(String name, int age) {  
        this(name, age, 0.0);  
    }  
}
```

✓ Constructor Chaining

```
public BankAccount(String accountNumber, String ownerName, String currency,  
double openingDeposit) {
```

```
    // all our previous code  
}
```

```
public BankAccount(String accountNumber, String ownerName, String currency) {  
    this(accountNumber, ownerName, currency, 0.0);  
}
```

```
public BankAccount(BankAccount migratedFromPartnerBank) {  
    this(  
        migratedFromPartnerBank.accountNumber,  
        migratedFromPartnerBank.ownerName,  
        migratedFromPartnerBank.currency,  
        migratedFromPartnerBank.balance  
    );  
}
```

✓ Controlling Access

- We made object construction safe
- Now, we need to focus on access and modification AFTER creation
- We can do this with the help of getters and setters

✓ Getters

Getters are methods used in Java to **access** the values of private variables

```
public double getBalance() {  
    return balance;  
}  
  
public String getAccountNumber() {  
    return accountNumber;  
}  
  
public String getOwnerName() {  
    return ownerName;  
}
```

✓ Setters

- ✎ Setters are methods used in Java to **modify** the values of private variables
- ✓ Do all attributes need a setter?

```
public void setOwnerName(String newName) {  
    ownerName = newName;  
}
```

✓ Setters

- Setters are methods used in Java to **modify** the values of private variables
- Do all attributes need a setter?

```
public void setOwnerName(String newName) {  
    if (newName == null || newName.isBlank()) {  
        throw new IllegalArgumentException("Owner name is required.");  
    }  
  
    ownerName = newName;  
}
```

Tying loose ends

```
public void deposit(double amount) {  
    if (amount <= 0.0) {  
        throw new IllegalArgumentException("Deposit must be positive.");  
    }  
  
    balance = balance + amount;  
}  
  
public void withdraw(double amount) {  
    if (amount <= 0.0) {  
        throw new IllegalArgumentException("Withdrawal must be positive.");  
    }  
  
    if (amount > balance) {  
        throw new IllegalStateException("Insufficient funds.");  
    }  
  
    balance = balance - amount;  
}
```

New Requirement

- Suppose, we have a field `openedOn`
- The bank wishes to know how long an account has been open
- How would we incorporate this?
- Should we store `ageInDays` as a field?
- What happens if a method forgets to update it?
- Storing derived/computed values is one of the most common sources of silent bugs

✓ Computed Fields

Anything that can be computed from existing state should be computed on demand

```
public long getAgeInDays() {  
    int ageInDays;  
  
    // calculate days between openedOn and today  
  
    return ageInDays;  
}
```

✓ Helper Methods

- As the class grows, validation logic has to be repeated again and again
- We can extract them into helper methods

```
public void deposit(double amount) {  
    if (amount <= 0.0) {  
        throw new IllegalArgumentException("Deposit must be positive.");  
    }  
  
    balance = balance + amount;  
}
```

Helper Methods

- As the class grows, validation logic has to be repeated again and again
- We can extract them into helper methods

```
private void requirePositive(double amount) {  
    if (amount <= 0.0) {  
        throw new IllegalArgumentException("Amount must be positive.");  
    }  
}  
  
public void deposit(double amount) {  
    requirePositive(amount);  
  
    balance = balance + amount;  
}
```

✓ Class Invariant

- ✓ ● An invariant is a condition that must always remain true for every object throughout its lifetime
- ✓ ● What conditions should always be true for a BankAccount object, no matter what operations happen on it?
 - ✓ ○ accountNumber must never be null or blank
 - ✓ ○ ownerName must never be null or blank
 - ✓ ○ currency must never be null or blank
 - ✓ ○ balance must always be greater than or equal to 0
 - ✓ ○ openedOn must never be a future date

Polymorphism

01

**What is
Polymorphism?**

02

**Types of
Polymorphism**

03

**Compile-Time
Polymorphism**

04

**Method & Operator
Overloading**

05

Real-Life Examples

✓ What is Polymorphism?

Poly → Many

Morph → Forms

- ✓ ● Polymorphism literally means "**many forms**"
- ✓ ● Allows objects, methods or operators to exhibit different behaviors under different circumstances

✓ Real-Life Analogy

Consider a **person** who adapts their behavior depending on context — yet remains the same individual:

- **Student in class**
Studies and attends classes
- **Son/Daughter at home**
Interacts with family members
- **Customer in a shop**
Browses, selects, and purchases goods

✓ Types of Polymorphism

Java supports two major categories of polymorphism. Each acts at a different point in the program's lifecycle.

- ✓ • Compile-Time Polymorphism/Static Polymorphism
- ✓ • Run-Time Polymorphism/Dynamic Polymorphism

Compile-Time Polymorphism

- ✓ Also called Static Polymorphism / Early Binding
- ✓ The entity is determined by the **compiler** — before the program ever runs
- Achieved through:
 - Method Overloading
 - Operator Overloading

✓ Method Overloading

Defining multiple methods with the **same name** inside the same class, each with a **different parameter list**

Methods must differ in at least one of the following:

- ✓ ■ Number of parameters
- ✓ ■ Data type of parameters
- ✓ ■ Order of parameters

Return type alone is **NOT** sufficient to overload a method

✓ Method Overloading

1

✓ Method Signature Analysis

The compiler reads the **number**, **type**, and **order of parameters** at every method call site

2

✓ Method Matching

It searches for a method whose signature matches the call. If found, it binds that method to the call

3

✓ Early Binding

The binding is permanent and happens at compile time

Example

Example 1: Different Number of Parameters

```
class Demo {  
    void display(int a) {  
        System.out.println("One number: " + a);  
    }  
  
    void display(int a, int b) {  
        System.out.println("Two numbers: " + a + " " + b);  
    }  
}
```

```
// display(5)      → calls first method  
// display(5, 10) → calls second method
```

Example

Example 2: Different Parameter Types

```
class Demo {  
    void display(int number) {  
        System.out.println("Integer: " + number);  
    }  
  
    void display(String text) {  
        System.out.println("String: " + text);  
    }  
}
```

```
// display(100)      → Integer: 100  
// display("Hello") → String: Hello
```

Example

Example 3: Different Parameter Order

```
class Demo {  
    void display(int a, String b) {  
        System.out.println(a + " " + b);  
    }  
  
    void display(String b, int a) {  
        System.out.println(b + " " + a);  
    }  
}
```

```
// print(1, "Hello") → 1 Hello  
// print("Hello", 1) → Hello 1
```

✓ Operator Overloading

- ✓ ● Operator overloading means the same operator performs different operations depending on the operand types
- ✓ ● In Java, user-defined operator overloading is not supported

Built-in Operator Overloading

While you cannot create your own, Java does have one primary example of built-in operator overloading:

The + Operator: This operator is overloaded to perform two different tasks:

1. **Arithmetic Addition:** Used for adding numeric types (e.g., $5 + 10$).
2. **String Concatenation:** Used for joining strings together (e.g., "Hello " + "World").

Benefits



Readability

Programs become intuitive. Calling `add()` always means `add` — regardless of the input type



Flexibility

Different implementations can exist, allowing the program to handle different inputs elegantly



Reduces Complexity

Eliminates the need to create separate method names for every operation